| | |
|---|---|
| **Name of Student: Minisha Singh** | **PRN: 25070122292** |
| **Semester: III** | **Year : AY 2026-27** |
| **Subject: Programming with Java** | |
| **ASSIGNMENT No : 10** | |
| **TITLE : Abstract Classes and Methods** | |

**Problem Statement**

Write a Java program to demonstrate the use of abstract classes and abstract methods

**Learning Objectives**

To implement abstraction using abstract classes and abstract methods.

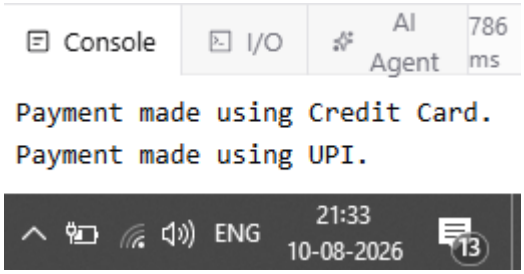
**Exercise**

**1.Create an abstract Payment class and implement Credit Card and UPI payment classes.?**

**Code:**

```
abstract class Payment {
abstract void makePayment();
}
class CreditCard extends Payment {
void makePayment() {
System.out.println("Payment made using Credit Card.");
}
}
class UPI extends Payment {
void makePayment() {
System.out.println("Payment made using UPI.");
}
}
public class Main {
public static void main(String[] args) {
Payment p1 = new CreditCard();
Payment p2 = new UPI();
p1.makePayment();
p2.makePayment();
}
}
```

**Output:**

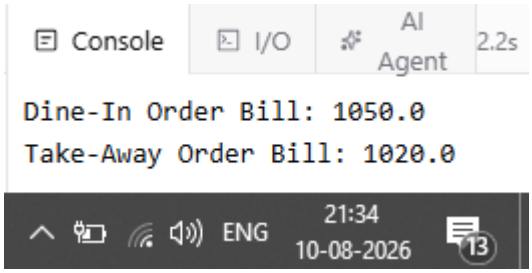



**2.Create an abstract FoodOrder class with an abstract method calculateBill(). Implement two subclasses, DineInOrder and TakeAwayOrder, to calculate and display the total bill based on the order type.**

**Code:**

```
abstract class FoodOrder {
double amount;
FoodOrder(double amount) {
this.amount = amount;
}
abstract void calculateBill();
}
class DineInOrder extends FoodOrder {
DineInOrder(double amount) {
super(amount);
}
void calculateBill() {
double total = amount + (amount * 0.05);
System.out.println("Dine-In Order Bill: " + total);
}
}
class TakeAwayOrder extends FoodOrder {
TakeAwayOrder(double amount) {
super(amount);
}
void calculateBill() {
double total = amount + (amount * 0.02);
System.out.println("Take-Away Order Bill: " + total);
}
}
public class Main {
public static void main(String[] args) {
FoodOrder dineIn = new DineInOrder(1000);
FoodOrder takeAway = new TakeAwayOrder(1000);
dineIn.calculateBill();
takeAway.calculateBill();
}
}
```

**Output:**




**Conclusion**

Abstraction is used to hide unnecessary implementation details and show only the important features of a program. Abstract classes provide a common structure for related classes, while abstract methods are implemented by subclasses. In this program, different payment and food order classes provide their own implementation of abstract methods. Thus, abstraction makes the program simple, organized, reusable, and easier to maintain.